

Chapter 3

Systems, Composition, and Transport

本章では文脈制限 (context restriction) の操作を定義する。これにより、立方体の「部分多面体 (sub-polyhedra)」に対応する新しい幾何学的形状が記述できるようになり、それを用いて合成の操作が導入される。合成は、homogeneous composition (hcomp) と移送 (transp) の2つの primitive として与えられ、これらから Path 型の除去規則が導かれる。

3.1 The Face Lattice

本節では、開いた箱の「側面の広がり」を語るための代数 $\mathbb{F}$ を導入する。構成は4段階である。まず束 $\mathbb{F}$ とその標準形を定義し (Section 3.1.1)、次に $\mathbb{I}$ と $\mathbb{F}$ を結ぶ写像たち—条件写像 ($r = \varepsilon$) と代入—を導入する (Section 3.1.2)。そのうえで文脈の restriction を定義し (Section 3.1.3)、制限された文脈における判断的等値性を、 $\mathbb{F}$ の元が定める合同として与える (Section 3.1.4)。最後の対応は、Section 3.2 以降のすべての側条件 (side condition) の読み方を定める基盤となる。

3.1.1 The Lattice $\mathbb{F}$

Definition 18 (The Face Lattice $\mathbb{F}$). *face lattice* $\mathbb{F}$ とは、関係 $(i = 0) \wedge (i = 1) = 0_{\mathbb{F}}$ を課して記号 $(i = 0)$ と $(i = 1)$ から生成される有界分配束である。 $\mathbb{F}$ の元を *face formula* と呼び、次の文法で記述される：

$$\varphi, \psi ::= 0_{\mathbb{F}} \mid 1_{\mathbb{F}} \mid (i = 0) \mid (i = 1) \mid \varphi \wedge \psi \mid \varphi \vee \psi$$

幾何学的には、face formula は立方体の「部分多面体」を記述する。たとえば $(i = 0) \vee (j = 1)$ は、正方形の方向 j の面と方向 i の面の合併とみなせる。この直観は、面への標準的な分解を与える次の補題で裏づけられる。

Lemma 19 (Disjunctive Normal Form). 相異なる名前についての生成元の有限個の交わり

$$(i_1 = \varepsilon_1) \wedge \cdots \wedge (i_k = \varepsilon_k) \quad (\varepsilon_1, \dots, \varepsilon_k \in \{0, 1\}; k = 0 \text{ のときは } 1_{\mathbb{F}})$$

を面 (face) と呼ぶ。\$\mathbb{F}\$ の任意の元は、\$0_{\mathbb{F}}\$ であるか、または有限個の面の結びに等しい。この表示により、\$\mathbb{F}\$ の等値性は決定可能である。

Proof. face formula に分配律を適用して選言標準形に直す。各連言項のうち、同じ名前の (\$i = 0\$) と (\$i = 1\$) が両方現れる項は定義関係により \$0_{\mathbb{F}}\$ となって消え、同じ生成元の重複は冪等律でまとめられるから、残る各項は面である。等値性の判定は次のとおり。面どうしについては、\$f \leq g\$ であることと「\$g\$ に現れる生成元がすべて \$f\$ にも現れる」ことが同値である。一般の元 \$\varphi = f_1 \vee \cdots \vee f_m\$ と \$\psi = g_1 \vee \cdots \vee g_n\$ については、\$\varphi \leq \psi\$ であることと「各 \$f_i\$ がある \$g_{i'}\$ 以下である」ことが同値であり (\$\Leftarrow\$ は結びの普遍性、\$\Rightarrow\$ は分配束において面が結び素 (join-prime) であることによる)、\$\varphi = \psi\$ は両方向の \$\leq\$ の成立と同値である。いずれも有限個の生成元の包含の確認に帰着するから、決定可能である。 \$\square\$

面は \$\mathbb{F}\$ の結び既約元 (join-irreducible element) にほかならず、Lemma 19 は「有限分配束の任意の元は、その下にある結び既約元の結びである」という一般的事実の \$\mathbb{F}\$ における実例である。名前の有限集合 \$I\$ に対し、\$I\$ の boundary \$\partial I\$ を、\$i \in I\$ を動く (\$i = 0\$) \$\vee\$ (\$i = 1\$) すべての選言として定義する。これは高々 \$I\$ の元にのみ依存する元のうち \$1_{\mathbb{F}}\$ 未満の最大元である。

3.1.2 The Maps from \$\mathbb{I}\$ to \$\mathbb{F}\$

\$\mathbb{I}\$ と \$\mathbb{F}\$ は次の 2 つの対応で結ばれる：\$\mathbb{I}\$ の元 \$r\$ を「\$r\$ の値が端点 \$\varepsilon\$ である」という名前たちへの条件 (\$r = \varepsilon\$) へ送る写像 \$\mathbb{I} \rightarrow \mathbb{F}\$ と、\$\mathbb{I}\$ の式の名前への代入 \$[r/i]\$ による face formula への作用である。後者は、写像としては \$\mathbb{F}\$ から \$\mathbb{F}\$ への束準同型だが、\$\mathbb{I}\$ の式 \$r\$ と名前 \$i\$ をパラメータとする族であり、\$\mathbb{I}\$ 側のデータが \$\mathbb{F}\$ を動かすという意味で両者を結び (第 3 の対応—\$\mathbb{F}\$ の元から \$\mathbb{I}\$ 上の合同へ—は Section 3.1.4 で与える)。

\$\mathbb{I}\$ の元 \$r\$ は名前たちの式であり、\$[0, 1]\$ 上の読みでは、各名前の指す位置を定めるごとに \$r\$ の値が定まる。そこで「\$r\$ の値が終端 1 である」「\$r\$ の値が始端 0 である」という名前たちへの条件は、\$\mathbb{F}\$ の元 (立方体の部分領域) を表すことになる。例：\$(i \sqcap j = 1) = (i=1) \wedge (j=1)\$ は「min の値が 1 になるのは両座標が 1 のとき」を表す。

その意味で、\$(r = 1)\$, \$(r = 0)\$ は face formula と呼ばれる。以下、これらを \$r\$ の構成に関する再帰で同時に定義する。名前 \$i\$ に対する \$(i = 1)\$, \$(i = 0)\$ は \$\mathbb{F}\$ の生成元そのものとし、その他の場合は次のとおりとする：

$$\begin{array}{ll} (0 = 1) \stackrel{def}{=} 0_{\mathbb{F}} & (0 = 0) \stackrel{def}{=} 1_{\mathbb{F}} \\ (1 = 1) \stackrel{def}{=} 1_{\mathbb{F}} & (1 = 0) \stackrel{def}{=} 0_{\mathbb{F}} \\ (1-r = 1) \stackrel{def}{=} (r = 0) & (1-r = 0) \stackrel{def}{=} (r = 1) \\ (r \sqcap s = 1) \stackrel{def}{=} (r = 1) \wedge (s = 1) & (r \sqcap s = 0) \stackrel{def}{=} (r = 0) \vee (s = 0) \\ (r \sqcup s = 1) \stackrel{def}{=} (r = 1) \vee (s = 1) & (r \sqcup s = 0) \stackrel{def}{=} (r = 0) \wedge (s = 0) \end{array}$$

各節は \$[0, 1]\$ 上の読み—min(\$r, s\$) が 1 であるのは \$r, s\$ がともに 1 のとき、min(\$r, s\$) が 0 であるのは少なくとも一方が 0 のとき、など—をそのまま形式化したものである。

Lemma 20. 上の定義は well-defined である。すなわち、 $\mathbb{I}$ の等式（束の公理と de Morgan 律）で等しい 2 つの式は、等しい face formula を定める。よって $r \mapsto (r = 1)$ は $\mathbb{I}$ から $\mathbb{F}$ への写像として定義され、有界束の準同型となる。

Proof. 束の公理（結合・交換・冪等・吸収・分配）については、上の節が $\sqcap, \sqcup$ を $\wedge, \vee$ ($= 0$ の側では役割を入れ替えて) へ準同型的に送るから、 $\mathbb{F}$ が分配束であることにより両辺は等しい face formula になる。確認を要するのは de Morgan 代数に固有の等式である。対合律 $1 - (1 - r) = r$ については $(1 - (1 - r) = 1) = ((1 - r) = 0) = (r = 1)$ であり、 $= 0$ の側も同様。de Morgan 律 $1 - (r \sqcap s) = 1 - r \sqcup 1 - s$ については、左辺の $= 1$ 側が $(r \sqcap s = 0) = (r = 0) \vee (s = 0)$ 、右辺の $= 1$ 側が $((1 - r) = 1) \vee ((1 - s) = 1) = (r = 0) \vee (s = 0)$ で一致し、残りも双対的に確認できる。準同型性は $0, 1, \sqcap, \sqcup$ に対する節そのものが述べている。 $\square$

なお、この写像は対合を保つ写像ではない— $\mathbb{F}$ に対合はない—が、対合は節 $(1 - r = 1) \stackrel{\text{def}}{=} (r = 0)$ により、 $(r = 1)$ と $(r = 0)$ の役割の交換として反映されている。

Lemma 21. 任意の $r : \mathbb{I}$ に対して、 $(r = 1) \wedge (r = 0) = 0_{\mathbb{F}}$ が成り立つ。

Proof. r の構成に関する帰納法による。 r が名前 i のときは、 $\mathbb{F}$ の定義関係 $(i = 0) \wedge (i = 1) = 0_{\mathbb{F}}$ そのものである。 r が $0, 1$ のときは $1_{\mathbb{F}} \wedge 0_{\mathbb{F}} = 0_{\mathbb{F}}$ 。 r が $1 - s$ のときは、2 つの成分が入れ替わるだけなので s の場合に帰着する。 r が $s \sqcap t$ のときは、分配律により

$$(s \sqcap t = 1) \wedge (s \sqcap t = 0) = ((s = 1) \wedge (t = 1) \wedge (s = 0)) \vee ((s = 1) \wedge (t = 1) \wedge (t = 0))$$

であり、帰納法の仮定によりどちらの項も $0_{\mathbb{F}}$ になる。 $s \sqcup t$ の場合は双対である。 $\square$

また、 $\mathbb{I}$ の式 r を名前 i へ代入する操作 $[r/i]$ の face formula への作用を、face formula の構成に関する再帰で次のように定義する：

$$\begin{array}{ll} (i = 1)[r/i] \stackrel{\text{def}}{=} (r = 1) & (i = 0)[r/i] \stackrel{\text{def}}{=} (r = 0) \\ (j = \varepsilon)[r/i] \stackrel{\text{def}}{=} (j = \varepsilon) \quad (j \neq i) & \\ 0_{\mathbb{F}}[r/i] \stackrel{\text{def}}{=} 0_{\mathbb{F}} & 1_{\mathbb{F}}[r/i] \stackrel{\text{def}}{=} 1_{\mathbb{F}} \\ (\varphi \wedge \psi)[r/i] \stackrel{\text{def}}{=} \varphi[r/i] \wedge \psi[r/i] & (\varphi \vee \psi)[r/i] \stackrel{\text{def}}{=} \varphi[r/i] \vee \psi[r/i] \end{array}$$

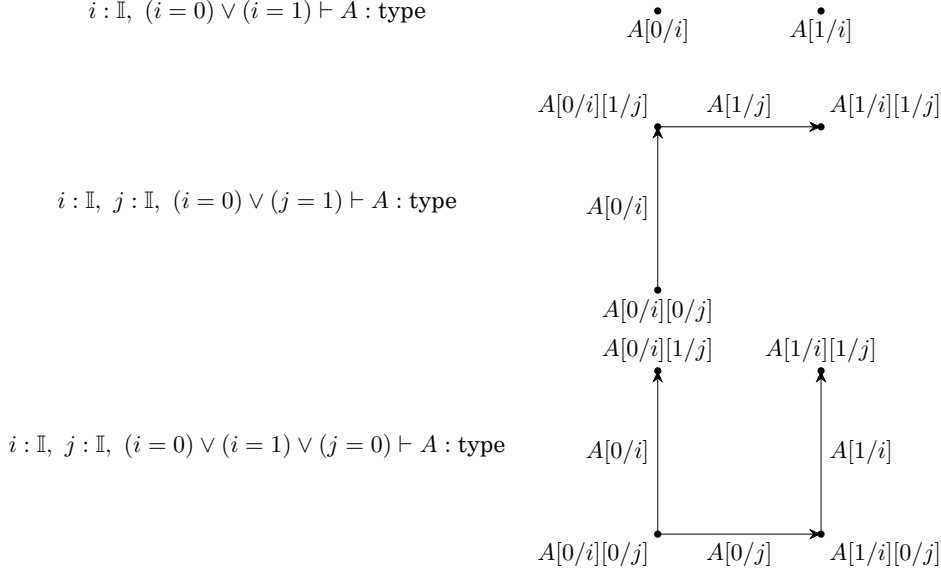
これは束準同型 $\mathbb{F} \rightarrow \mathbb{F}$ である。とくに、面 $f = (i_1 = \varepsilon_1) \wedge \cdots \wedge (i_k = \varepsilon_k)$ をなす各生成元は、対応する代入 $[\varepsilon_1/i_1] \cdots [\varepsilon_k/i_k]$ のもとで $1_{\mathbb{F}}$ に移るから、 f 自身も $1_{\mathbb{F}}$ に移る (Section 3.1.4 で用いる)。

3.1.3 Restriction of Contexts

$\Gamma \vdash \psi : \mathbb{F}$ は、 ψ が Γ で宣言された名前のみを用いる face formula であることを意味する。文脈に対する新しい操作として *restriction* を導入する：

$$\begin{array}{c} \Gamma, \Delta \quad ::= \quad \dots \mid \Gamma, \varphi \\ \hline \Gamma \vdash \varphi : \mathbb{F} \\ \Gamma, \varphi \text{ context} \end{array}$$

restriction とは、文脈に face formula を加える操作であり、これにより新しい幾何学的形状が記述できる。前述のとおり文脈 $\Gamma = i : \mathbb{I}, j : \mathbb{I}$ における型は正方形の上で定義されているが、制限された文脈 Γ, φ における型はこの正方形の面の、互いに compatible な合併の上で定義される：



3.1.4 Judgmental Equality under Restriction

Section 3.1.2 の写像たちと逆向きに、 $\mathbb{F}$ の各元を $\mathbb{I}$ 上の合同に対応させることができる。ここで $\mathbb{I}$ 上の合同 (congruence on $\mathbb{I}$) とは、 $\mathbb{I}$ の演算と両立する同値関係、すなわち $r \sim s$ かつ $r' \sim s'$ ならば $r \sqcap r' \sim s \sqcap s'$ 、 $r \sqcup r' \sim s \sqcup s'$ 、 $1-r \sim 1-s$ が成り立つような同値関係 $\sim$ のことである。合同の全体は交わりについて閉じており、包含順序で束をなす (congruence lattice)。

対応は、Lemma 19 の表示を用いて定義する。まず、面 $f = (i_1 = \varepsilon_1) \wedge \cdots \wedge (i_k = \varepsilon_k)$ に対し、対応する代入を

$$\sigma_f \stackrel{\text{def}}{=} [\varepsilon_1/i_1] \cdots [\varepsilon_k/i_k]$$

と定める (名前が相異なるので順序に依らない。 $k = 0$ 、すなわち $f = 1_{\mathbb{F}}$ のときは恒等代入とする)。そのうえで、面の結び $\psi = f_1 \vee \cdots \vee f_m$ と $\mathbb{I}$ の元 r, s に対し— $\mathbb{I}$ の元とは、Definition 11 の文法で名前たちから組み立てられる式であった。名前を変数とする「多項式」であり、各名前の指す位置を定めるとにその値が定まるから、 σ_f による複数の名前への同時代入は、多変数多項式のいくつかの変数に値を入れる操作にあたり、幾何学的には r を面 f 上へ制限することである—次のように定める：

$$r = s \pmod{\psi} \quad \text{iff} \quad \text{すべての } l \text{ について } r\sigma_{f_l} = s\sigma_{f_l}$$

$m = 0$ 、すなわち $\psi = 0_{\mathbb{F}}$ のときは条件が空になるので、 $r = s \pmod{0_{\mathbb{F}}}$ はつねに成り立つ (すべての元を同一視する自明な合同)。たとえば ψ が面 $(i = 0) \wedge (j = 1)$ なら、 $r = s \pmod{\psi}$ は $r[0/i][1/j] = s[0/i][1/j]$ と同値である。

Lemma 22. 上の定義は well-defined である。すなわち、関係 $r = s \pmod{\psi}$ は ψ の面の結びとしての表示の取り方に依らない。さらに、各 $r = s \pmod{\psi}$ は $\mathbb{I}$ 上の合同であり、写像 $\psi \mapsto (\text{mod } \psi)$ は順序を逆にする—とくに $\mathbb{F}$ の結びを合同の交わりに送る。

Proof. 代入 σ_f は de Morgan 代数の準同型 $\mathbb{I} \rightarrow \mathbb{I}$ であるから、関係 $r\sigma_f = s\sigma_f$ はその核 (kernel) として合同であり、合同の有限個の交わりはまた合同である。順序について：面どうしでは、 $f \leq g$ であることは「 g の生成元がすべて f にも現れる」ことと同値であった (Lemma 19 の証明)。このとき σ_f は σ_g にさらに代入を続けたものだから、 $r\sigma_g = s\sigma_g$ ならば $r\sigma_f = s\sigma_f$ である。すなわち $f \leq g$ ならば $(\text{mod } g)$ の合同は $(\text{mod } f)$ の合同に含まれる。一般の $\psi \leq \psi'$ については、 ψ の各面がある ψ' の面以下であること (同証明) から、交わりの包含が従う。well-definedness はその特別な場合である：同じ元を与える 2 つの表示は互いに $\leq$ で支配し合うから、定まる合同も互いに含み合い、一致する。 $\square$

Lemma 23. 関係 $r = s \pmod{(i = \varepsilon)}$ は、 i と ε を同一視する最小の合同である： $i \sim \varepsilon$ を満たす任意の合同 $\sim$ について、 $r = s \pmod{(i = \varepsilon)}$ ならば $r \sim s$ が成り立つ。とくに $(\text{mod } (i = 1))$ のもとでは $1-i = 0$ も成り立つ。

Proof. まず $i[\varepsilon/i] = \varepsilon = \varepsilon[\varepsilon/i]$ より $i = \varepsilon \pmod{(i = \varepsilon)}$ である。最小性： $\sim$ を $i \sim \varepsilon$ なる合同とする。任意の r について $r \sim r[\varepsilon/i]$ が、 r の構成に関する帰納法で従う (r が名前 i のときは仮定そのもの、他の名前と $0, 1$ のときは反射律、演算のときは合同性による)。よって $r[\varepsilon/i] = s[\varepsilon/i]$ ならば $r \sim r[\varepsilon/i] = s[\varepsilon/i] \sim s$ である。後半は、 $i = 1 \pmod{(i = 1)}$ の両辺に $1-(\cdot)$ を施せば—合同は $1-(\cdot)$ と両立するから— $1-i = 1-1 = 0 \pmod{(i = 1)}$ を得る。 $\square$

各文脈 Γ には $\mathbb{I}$ 上の合同が再帰的に対応する： Γ, ψ の合同は Γ の定める合同と ψ の定める合同の結びであり、 $()$ の定める合同は $\mathbb{I}$ 上の等値性、拡張 $x : A$ や $i : \mathbb{I}$ は合同を変えない。判断 $\Gamma \vdash r = s : \mathbb{I}$ は、 $r = s \pmod{\Gamma}$ かつ $\Gamma \vdash r : \mathbb{I}$ かつ $\Gamma \vdash s : \mathbb{I}$ を意味する (Section 2.1 の判断的等値性の再定義)。 Γ が restriction を含まなければ、これは $\mathbb{I}$ における $r = s$ を意味する。 i が Γ で宣言されていれば、 $\Gamma, (i = 0) \vdash r = s : \mathbb{I}$ は $\Gamma \vdash r[0/i] = s[0/i] : \mathbb{I}$ と同値である ($r = s \pmod{(i = 0)}$ の定義そのものである)。

$\mathbb{F}$ 上の等値性判断も、同じ機構で局所化される。 $\mathbb{F}$ の元どうしの関係を、 ψ の面表示 $f_1 \vee \cdots \vee f_m$ と face formula への代入 (Section 3.1.2) を用いて

$$\varphi_0 = \varphi_1 \pmod{\psi} \quad \text{iff} \quad \text{すべての } l \text{ について } \varphi_0 \sigma_{f_l} = \varphi_1 \sigma_{f_l}$$

と定める。well-definedness と合同性は Lemma 22 と同じ論法 (σ_{f_l} は束準同型 $\mathbb{F} \rightarrow \mathbb{F}$ であるからその核は合同、合同の有限交わりは合同) で従い、 $m = 0$ ($\psi = 0_{\mathbb{F}}$) のとき条件が空になってすべての元が同一視される—とくに $0_{\mathbb{F}} = 1_{\mathbb{F}} \pmod{0_{\mathbb{F}}}$ —ことも $\mathbb{I}$ のときと同様である。各文脈 Γ は、 $\mathbb{I}$ のときと同じ再帰で $\mathbb{F}$ 上の合同を定め、判断 $\Gamma \vdash \varphi_0 = \varphi_1 : \mathbb{F}$ はその合同のもとでの一致を意味する。この合同は次の特徴づけをもつ。

Lemma 24. 任意の $\varphi_0, \varphi_1, \psi \in \mathbb{F}$ について

$$\varphi_0 = \varphi_1 \pmod{\psi} \quad \text{iff} \quad \psi \wedge \varphi_0 = \psi \wedge \varphi_1 \quad (\mathbb{F} \text{ における等式})$$

が成り立つ。したがって $\Gamma, \psi \vdash \varphi_0 = \varphi_1 : \mathbb{F}$ は $\Gamma \vdash \psi \wedge \varphi_0 = \psi \wedge \varphi_1 : \mathbb{F}$ と同値である。

Proof. $\psi \wedge \varphi = (f_1 \wedge \varphi) \vee \cdots \vee (f_m \wedge \varphi)$ (分配律) であり、 $\psi \wedge \varphi_0 = \psi \wedge \varphi_1$ の両辺と各 f_l との交わりをとれば $f_l \wedge \varphi_0 = f_l \wedge \varphi_1$ が回収できる ($f_l \leq \psi$ による) から、面ごとの同値

$$f \wedge \varphi_0 = f \wedge \varphi_1 \quad \text{iff} \quad \varphi_0 \sigma_f = \varphi_1 \sigma_f$$

を示せばよい。(⇒) 両辺に束準同型 σ_f を施す。Section 3.1.2 末尾の観察により $f \sigma_f = 1_{\mathbb{F}}$ だから、 $\varphi_0 \sigma_f = (f \wedge \varphi_0) \sigma_f = (f \wedge \varphi_1) \sigma_f = \varphi_1 \sigma_f$ を得る。(⇐) まず恒等式 $f \wedge \varphi = f \wedge (\varphi \sigma_f)$ が任意の φ について成り立つ：両辺は φ について束準同型だから生成元 ($i = \varepsilon$) で確かめればよく、 f が ($i = \varepsilon$) を含むなら両辺とも f 、 f が ($i = 1 - \varepsilon$) を含むなら定義関係 ($i = 0$) $\wedge$ ($i = 1$) = $0_{\mathbb{F}}$ により両辺とも $0_{\mathbb{F}}$ 、 i が f に現れないなら σ_f は生成元を変えない。これを用いて $f \wedge \varphi_0 = f \wedge (\varphi_0 \sigma_f) = f \wedge (\varphi_1 \sigma_f) = f \wedge \varphi_1$ を得る。□

とくに φ_1 として $1_{\mathbb{F}}$ をとると、 $\Gamma, \psi \vdash \varphi = 1_{\mathbb{F}} : \mathbb{F}$ は $\Gamma \vdash \psi \wedge \varphi = \psi : \mathbb{F}$ 、すなわち Γ の合同のもとでの $\psi \leq \varphi$ と同値である。つまり制限文脈での判断「 $\varphi = 1_{\mathbb{F}}$ 」は「extent ψ の上では φ は真」という局所化された主張であって、 φ が $\mathbb{F}$ の最大元に絶対的に等しいという主張ではない^{*1}。たとえば $i : \mathbb{I}$, $(i = 0) \vee (i = 1) \vdash (i = 0) \vee (i = 1) = 1_{\mathbb{F}} : \mathbb{F}$ である。この読み方は、Section 3.2 以降のすべての側条件の理解にそのまま効いてくる。

3.1.5 Quantifier Elimination

$\mathbb{F}$ 上には単純な量子化消去がある：名前 i に対し、 $\forall i : \mathbb{F} \rightarrow \mathbb{F}$ を、($i = 0$) と ($i = 1$) を $0_{\mathbb{F}}$ に送り、他の生成元上では恒等であるような束準同型として定義する。 ψ が i に依存しないとき、

$$\psi \leq \varphi \quad \text{iff} \quad \psi \leq \forall i. \varphi$$

が成り立つ。たとえば φ が $(i = 0) \vee ((i = 1) \wedge (j = 0)) \vee (j = 1)$ なら、 $\forall i. \varphi$ は $(j = 1)$ である。この操作は、Chapter 5 で導入する Glue 型の合成の定義において本質的な役割を果たす。

$\mathbb{F}$ はブール代数ではないので、一般には $\varphi = (\varphi \wedge (i = 0)) \vee (\varphi \wedge (i = 1))$ は成り立たないが、つねに次の分解が成り立つ：

Lemma 25. $\mathbb{F}$ の任意の元 φ と任意の名前 i に対して

$$\varphi = (\forall i. \varphi) \vee (\varphi \wedge (i = 0)) \vee (\varphi \wedge (i = 1))$$

が成り立つ。また $\varphi \wedge (i = 0) \leq \varphi[0/i]$ および $\varphi \wedge (i = 1) \leq \varphi[1/i]$ が成り立つ。

^{*1} 局所化されているのは等値性判断の方であり、定数 $1_{\mathbb{F}}$ の指すものはつねに $\mathbb{F}$ の最大元である。商で読むこともできる： $(\text{mod } \psi)$ による $\mathbb{F}$ の商は主イデアル $\downarrow \psi$ と同型で $(\varphi \mapsto \psi \wedge \varphi)$ 、そこでの $1_{\mathbb{F}}$ の像 (= 商の頂) はちょうど ψ である。

3.2 Syntax and Inference Rules for Systems

system は、「部分多面体」を立方体の compatible な合併として導入する構文である。path 型を備えた依存型理論の構文への拡張は次のとおりである：

$$t, u, A, B ::= \dots \mid [\varphi_1 t_1, \dots, \varphi_n t_n] \quad (\text{system})$$

$n = 0$ も許され、そのとき空 system $[]$ を得る。

Definition 26 (Inference Rules for Systems).

$$\frac{\Gamma, \varphi_1 \vdash A_1 : \text{type} \quad \dots \quad \Gamma, \varphi_n \vdash A_n : \text{type} \quad \Gamma, \varphi_k \wedge \varphi_m \vdash A_k = A_m : \text{type} \quad (1 \leq k, m \leq n)}{\Gamma \vdash [\varphi_1 A_1, \dots, \varphi_n A_n] : \text{type}}$$

$$\frac{\Gamma \vdash A : \text{type} \quad \Gamma, \varphi_1 \vdash t_1 : A \quad \dots \quad \Gamma, \varphi_n \vdash t_n : A \quad \Gamma, \varphi_k \wedge \varphi_m \vdash t_k = t_m : A \quad (1 \leq k, m \leq n)}{\Gamma \vdash [\varphi_1 t_1, \dots, \varphi_n t_n] : A}$$

$$\frac{\Gamma, \varphi_1 \vdash J \quad \dots \quad \Gamma, \varphi_n \vdash J}{\Gamma \vdash J} \quad \frac{\Gamma \vdash [\varphi_1 A_1, \dots, \varphi_n A_n] : \text{type} \quad \Gamma \vdash \varphi_k = 1_{\mathbb{F}} : \mathbb{F}}{\Gamma \vdash [\varphi_1 A_1, \dots, \varphi_n A_n] = A_k : \text{type}}$$

$$\frac{\Gamma \vdash [\varphi_1 t_1, \dots, \varphi_n t_n] : A \quad \Gamma \vdash \varphi_k = 1_{\mathbb{F}} : \mathbb{F}}{\Gamma \vdash [\varphi_1 t_1, \dots, \varphi_n t_n] = t_k : A}$$

各規則には側条件 $\Gamma \vdash \varphi_1 \vee \dots \vee \varphi_n = 1_{\mathbb{F}} : \mathbb{F}$ が付く。

この側条件は Γ が十分に制限されていることを要求する：たとえば $\Delta, (i = 0) \vee (i = 1) \vdash (i = 0) \vee (i = 1) = 1_{\mathbb{F}} : \mathbb{F}$ である (Lemma 24 とその直後の議論のとおり、この判断は「 Γ の extent が面の合併に覆われている」ことを述べる)。第 1 の規則は型の system を導入する。各 A_k は φ_k 上で定義され、重なるところでは型が一致することが要求される。第 2 の規則は項に対する同様の規則である。第 3 の規則は、各 φ_k において局所的に成り立つ判断は大域的に成り立つことを述べる（特に $n = 0$ も許され、そのとき側条件は $\Gamma \vdash 0_{\mathbb{F}} = 1_{\mathbb{F}} : \mathbb{F}$ となる）。最後の 2 つの規則は system が正しい面をもつことを保証する。なお $n = 0$ のとき第 2 の規則は「 $\Gamma \vdash 0_{\mathbb{F}} = 1_{\mathbb{F}} : \mathbb{F}$ かつ $\Gamma \vdash A : \text{type}$ ならば $\Gamma \vdash [] : A$ 」と読む。

代入判断は次で拡張される： $\Delta \vdash \sigma : \Gamma, \varphi$ とは、 $\Delta \vdash \sigma : \Gamma$ かつ $\Gamma \vdash \varphi : \mathbb{F}$ かつ $\Delta \vdash \varphi \sigma = 1_{\mathbb{F}} : \mathbb{F}$ のことである。

制限文脈での判断 $\Gamma, \varphi \vdash u : A$ が成り立つとき、 u を「extent」 φ 上で定義された部分元 (partial element) と呼ぶ。このとき、2 つの判断

$$\Gamma \vdash a : A \quad \Gamma, \varphi \vdash a = u : A$$

を満たす元 a は、部分元 u が全体へ延長可能 (extensible) であることの witness とみなせる^{*2}。以下、この「型割り当て+制限文脈での等値性」という判断の組が繰り返し現れ

^{*2} 原論文はこの 2 つの判断の組を $a : A[\varphi \mapsto u]$ と略記し、複数の制約をもつ場合も $a : A[\varphi_1 \mapsto u_1, \dots, \varphi_k \mapsto u_k]$ (各 φ_l 上で u_l に等しい) と書く。本書では、型割り当てと制限文脈での判断的等値性という異なる 2 つの判断を 1 つの型表現に畳み込むこの略記は採らず、つねに両者を書き分ける。原論文を並読する際は、この略記が現れるたびに上の 2 判断に読み戻せばよい。

る。たとえば $\Gamma, i : \mathbb{I} \vdash A : \text{type}$ かつ $\Gamma, i : \mathbb{I}, \varphi \vdash u : A$ ($\varphi = (i = 0) \vee (i = 1)$) のとき、部分元 u は 2 つの元 $\Gamma \vdash a_0 : A[0/i]$ と $\Gamma \vdash a_1 : A[1/i]$ で決まり、 $\Gamma, i : \mathbb{I} \vdash a : A$ かつ $a[0/i] = a_0, a[1/i] = a_1$ を満たす元 a は、 a_0 と a_1 を結ぶ path を与える。

Lemma 27. 次の規則は許容可能 (admissible) である：

$$\frac{\Gamma \vdash \varphi \leq \psi : \mathbb{F} \quad \Gamma, \psi \vdash J}{\Gamma, \varphi \vdash J} \quad \frac{\Gamma, 1_{\mathbb{F}} \vdash J}{\Gamma \vdash J} \quad \frac{\Gamma, \varphi, \psi \vdash J}{\Gamma, \varphi \wedge \psi \vdash J}$$

さらに、 φ が i に依存しないとき次の規則が許容可能であり、

$$\frac{\Gamma, i : \mathbb{I}, \varphi \vdash J}{\Gamma, \varphi, i : \mathbb{I} \vdash J}$$

これより一般に次が成り立つ：

$$\frac{\Gamma, i : \mathbb{I}, \varphi \vdash J}{\Gamma, \forall i. \varphi, i : \mathbb{I} \vdash J}$$

Remark 28 (Where the Extent Lives). system の項そのものは、extent を構文の一部としては持たない。extent を供給するのは周囲の文脈である：Definition 26 の側条件は、Lemma 24 により、文脈 Γ_0, ψ においては「 Γ_0 の合同のもとで $\psi \leq \varphi_1 \vee \dots \vee \varphi_n$ 」に展開される。すなわち system は、面の合併が周囲の extent を覆うときに、その extent 上の部分元として型付けされる。等式 $\psi = \varphi_1 \vee \dots \vee \varphi_n$ ではなく被覆 $\leq$ であることが要点で、これにより型付けは制限の単調性 (Lemma 27) や代入のもとで保たれる——文脈をさらに制限しても、同じ system の項が同じ型をもち続ける。なお、extent を型の側面に書き込む設計もある：Cubical Agda (Vezzosi et al., 2019) は partiality を型構成子 $\text{Partial } \varphi A$ として内部化し、system はその元として型付けされる。extent を文脈に置くか (原論文と本書)、型に書くか (Cubical Agda) は、同じ内容の 2 つの構文設計である。

3.3 Homogeneous Composition

Section 2.3 の末尾で述べたとおり、path 型の除去規則 (J に対応する規則) を正当化するには、合成の操作を体系に追加する必要がある。本書では、合成を 2 つの operation に分けて導入する：型が方向に沿って変化しない *homogeneous composition* (hcomp 、本節) と、型の線に沿って元を運ぶ移送 (*transport*; transp 、次節) である^{*3}。構文への拡張は次のとおりである：

$$t, A, B \quad ::= \quad \dots \mid \text{hcomp}_A^i [\varphi \mapsto u] u_0 \mid \text{transp}^i A \varphi u_0 \quad (\text{composition operations})$$

ここで u は extent φ 上の部分元である (Section 3.2)^{*4}。

^{*3} この分解は Coquand et al. (2018) に由来し、Cubical Agda (Vezzosi et al., 2019) の primitive の構成とも一致する。原論文 (Cohen et al., 2018) は、この 2 つを併せた一般の合成 comp を primitive として採用している。本章では comp は Section 3.5 で派生的操作として導入する (History and Further Reading も参照)。なお「線に沿って元を運ぶ」という比喻の妥当性については Remark 40 で論じる。

^{*4} 原論文 (Cohen et al. 2018, §4.4) はこの箇所を “a system on the extent φ ” と書くが、規則が課するのは制限文脈での型付けのみであり、同論文 §4.2 および Coquand et al. (2018) の用語では部分元にあたる。system の項が u の位置に現れるのは、複数の面から側面を与える略記 (後述) を展開したときである。なお部分元は項の構文範疇ではなく判断に相対的な役割であり、system の項もまた、その側条件を満たす制限文脈で型付けされて部分元となる (Remark 28)。

Definition 29 (Homogeneous Composition).

$$\frac{\Gamma \vdash \varphi : \mathbb{F} \quad \Gamma \vdash A : \text{type} \quad \Gamma, \varphi, i : \mathbb{I} \vdash u : A \quad \Gamma \vdash u_0 : A \quad \Gamma, \varphi \vdash u_0 = u[0/i] : A}{\Gamma \vdash \text{hcomp}_A^i [\varphi \mapsto u] u_0 : A}$$

さらに、結果は extent φ 上で側面の終端に等しい (∂ -等値性) :

$$\Gamma, \varphi \vdash \text{hcomp}_A^i [\varphi \mapsto u] u_0 =_{\partial} u[1/i] : A$$

hcomp_A^i は u の中の i を束縛する。型 A は文脈 Γ で型付けされており、方向 i に依存しない (homogeneous の名はこのことによる)。また、側面を複数の面とその上の部品から与えるときのために、次の略記を導入する :

$$[\varphi_1 \mapsto u_1, \dots, \varphi_n \mapsto u_n] \stackrel{\text{def}}{=} [\varphi_1 \vee \dots \vee \varphi_n \mapsto [\varphi_1 \ u_1, \dots, \varphi_n \ u_n]] \quad [] \stackrel{\text{def}}{=} [0_{\mathbb{F}} \mapsto []]$$

右辺の内側に現れるのが system の項である ($n = 0$ のときは空 system)。この略記のもとで、Definition 29 の前提と ∂ -等値性からは次の補題が導かれる。

Lemma 30 (Homogeneous Composition with Multiple Faces). すべての $1 \leq k, m \leq n$ について

$$\Gamma \vdash A : \text{type} \quad \Gamma \vdash \varphi_k : \mathbb{F} \quad \Gamma, \varphi_k, i : \mathbb{I} \vdash u_k : A \\ \Gamma, \varphi_k \wedge \varphi_m, i : \mathbb{I} \vdash u_k = u_m : A \quad \Gamma \vdash u_0 : A \quad \Gamma, \varphi_k \vdash u_0 = u_k[0/i] : A$$

が成立するとき、

$$\Gamma \vdash \text{hcomp}_A^i [\varphi_1 \mapsto u_1, \dots, \varphi_n \mapsto u_n] u_0 : A$$

が成り立ち、また各 k について ∂ -等値性

$$\Gamma, \varphi_k \vdash \text{hcomp}_A^i [\varphi_1 \mapsto u_1, \dots, \varphi_n \mapsto u_n] u_0 =_{\partial} u_k[1/i] : A$$

が成り立つ。

Proof. $\varphi \stackrel{\text{def}}{=} \varphi_1 \vee \dots \vee \varphi_n$ 、 $u \stackrel{\text{def}}{=} [\varphi_1 \ u_1, \dots, \varphi_n \ u_n]$ とおく (略記の右辺)。

u の型付け 制限文脈 $\Gamma, \varphi, i : \mathbb{I}$ では側条件 $\varphi_1 \vee \dots \vee \varphi_n = 1_{\mathbb{F}}$ が成り立つ (Lemma 24 によりこの判断は $\varphi \wedge (\varphi_1 \vee \dots \vee \varphi_n) = \varphi$ に展開され、 $\varphi \stackrel{\text{def}}{=} \varphi_1 \vee \dots \vee \varphi_n$ だから冪等律そのものである) から、項の system の規則 (Definition 26) が適用でき、前提の型付けと重なりでの一致から $\Gamma, \varphi, i : \mathbb{I} \vdash u : A$ を得る。

底面接続 各 k について、まず $\Gamma, \varphi, i : \mathbb{I} \vdash u : A$ から、代入 $[0/i]$ と制限の単調性 ($\varphi_k \leq \varphi$; Lemma 27) により $\Gamma, \varphi_k \vdash u[0/i] : A$ を得る。制限文脈 Γ, φ_k では $\varphi_k = 1_{\mathbb{F}}$ が成り立つから、system の面の規則により $u[0/i] = u_k[0/i]$ であり (φ_k は i を含まないので、代入 $[0/i]$ は system の各部品に分配される)、前提とあわせて $\Gamma, \varphi_k \vdash u_0 = u[0/i] : A$ を得る。 $\varphi \wedge \varphi_k = \varphi_k$ に注意すれば、Lemma 27 (制限の併合) によりこれは $\Gamma, \varphi, \varphi_k \vdash u_0 = u[0/i] : A$ と同じことであり、system の局所大域規則 (Definition 26 第 3 規則; 側条件は制限文脈 Γ, φ で成立) により $\Gamma, \varphi \vdash u_0 = u[0/i] : A$ が従う。以上で Definition 29 の前提がすべて揃い、 $\Gamma \vdash \text{hcomp}_A^i [\varphi \mapsto u] u_0 : A$ 、すなわち略記により主張の型付けを得る。

∂ -等値性 Definition 29 により $\Gamma, \varphi \vdash \text{hcomp}_A^i [\varphi \mapsto u] u_0 =_{\partial} u[1/i] : A$ であり、 $\varphi_k \leq \varphi$ だから Lemma 27 により Γ, φ_k でも成り立つ。そこでは $\varphi_k = 1_{\mathbb{F}}$ より、system の面の規則で $u[1/i] = u_k[1/i]$ である。

□

直観的には、 hcomp は型 A の中の開いた箱 (open box) —底面と側面はあるが蓋のない箱—に蓋 (lid) を与える操作である。 u_0 が底面を、 u が側面を指定する。この箱の「横」の方向を張るのは、規則中に明示されない文脈 Γ の側の名前たちであり、「縦」の方向を張るのは hcomp の束縛する fresh な名前 i である。以下、このことを規則の前提から順に読み取っていきう。

$$\Gamma \vdash u_0 : A$$

まず、 u_0 に関する上記の条件を読み取ろう。Section 2.1 で述べたとおり、名前を含む文脈で型付けされた項はそれ自身立方体であるから、上の判断の u_0 は一般には点ではなく、 Γ 中の名前たちが張る立方体—横方向いっぱい広がる面—である。そして u_0 は Γ で型付けされる以上、縦方向の名前 i を含みえない。したがって u_0 は i 方向に定数であり、 u_0 の縦方向の位置は指定されていない。 u_0 を $i = 0$ の「底面」の位置に据えるのは、後述の接続の前提である。

$$\Gamma, \varphi, i : \mathbb{I} \vdash u : A$$

次に、上記の条件は u が「側面」であることを表している。 u は、横方向には底面全体ではなく、face formula φ の切り出す部分 (Section 3.1) の上のみ定義され、他方 u_0 と違って i を含むうから、縦方向には方向 i に沿って箱の高さいっぱいになる—これが側面 (sides) である。そして u は extent φ 上の system、すなわち面ごとの部品 $u_1, \dots, u_n$ を貼り合わせた項であるから、その定義 (Definition 26) により、side とうしは重なるところで一致し、部品の台の合併は extent φ を覆い尽くす (側条件 $\varphi_1 \vee \dots \vee \varphi_n = 1_{\mathbb{F}}$ が制限文脈で成り立つ)。すなわち、側面は extent の範囲では隙間なく貼り合わさっている—ただし箱そのものは横に開いていてよい (Γ で $\varphi < 1_{\mathbb{F}}$ でありうる)。

$$\Gamma, \varphi \vdash u_0 = u[0/i] : A$$

さらに、上記の条件は、底面と側面が接続していることを表している。側面の定義されている範囲 φ の上で、その $i = 0$ 端が u_0 に一致している、と読める。「 u_0 は箱の底面 (bottom) を指定する」といえるのは、この判断的等値性によって u_0 が側面の $i = 0$ 端に貼り付けられるからである。つまり底面とは、 Γ -方向の代入で得られる面のことでなく、新しい方向 i の 0 端のことである。

以上が「開いた箱」である。 hcomp の与える蓋とは、底面と同じく Γ 全体の上で定義された面である (hcomp の項全体も Γ で型付けされ、 i を含まない)。これを $i = 1$ の端に据えるのが defbox 内の ∂ -等値性であり、蓋が extent φ 上で側面の終端 $u[1/i]$ に接続していることを表す—底面を $i = 0$ の端に据えた接続の前提と対をなす。たとえば Γ が名前 j を 1 つ含み、 $\varphi = (j = 0) \vee (j = 1)$ 、 $u = [(j = 0) u_1, (j = 1) u_2]$ —略記では $[(j = 0) \mapsto u_1, (j = 1) \mapsto u_2]$ —のとき、箱は正方形である：底面 u_0 は方向 j の線、側面 u_1, u_2 はその両端点から方向 i に伸びる 2 本の線、蓋はそれらの終端を結ぶ方向 j の線となる (後述の path 合成の図がまさにこの形である。ただしそこでは横方向の名前が

i 、束縛される縦方向の名前が j と、ここでの説明とは役割が逆であることに注意)。特に $\varphi = 1_{\mathbb{F}}$ のとき、Lemma 27 により制限は外れて $\Gamma \vdash \mathbf{hcomp}_A^i [1_{\mathbb{F}} \mapsto u] u_0 = u[1/i] : A$ となる。

文脈間の代入 $\Delta \vdash \sigma : \Gamma$ に対しては

$$(\mathbf{hcomp}_A^i [\varphi \mapsto u] u_0) \sigma = \mathbf{hcomp}_{A\sigma}^j [\varphi \sigma \mapsto u \sigma [j/i]] u_0 \sigma$$

が成り立つ (j は Δ に対して fresh; $\mathbf{hcomp}^i$ は i を束縛するから、 σ を施したうえで束縛名を fresh な j へ付け替える)。これは意味論的には合成操作の一意性 (uniformity) に対応する。

Example 31 (Transitivity of Path Types). $\mathbf{hcomp}$ を用いると、path 型の推移性が正当化できる: $p : \text{Path } A a b$ と $q : \text{Path } A b c$ に対して、合成 path を

$$p \cdot q \stackrel{\text{def}}{=} \langle i \rangle \mathbf{hcomp}_A^j [(i=0) \mapsto a, (i=1) \mapsto q j] (p i)$$

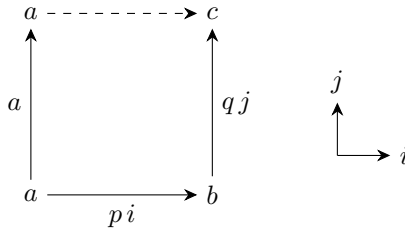
と定義し、 $p \cdot q : \text{Path } A a c$ を導出しよう。検算すべきことは2つある。第一に、 $\mathbf{hcomp}$ がそもそも適用できること、すなわち Lemma 30 の前提が要求する「側面と底面の接続」である。面 $(i=0)$ の上では、側面 a は j に依存しないからその始端も a 、底面は $p i = p 0 =_{\partial} a$ となって一致し、面 $(i=1)$ の上では、側面の始端は $q 0 =_{\partial} b$ 、底面は $p i = p 1 =_{\partial} b$ となって一致する (なお2つの側面の重なりは $(i=0) \wedge (i=1) = 0_{\mathbb{F}}$ だから、両立の前提は自明に満たされる)。第二に、端点の検算である。funext のとき (Section 2.2) と同じく、(PathI) がこの path 抽象に与える型は本体の面を端点とする path 型であるから、両面が $a \cdot c$ に判断的に等しいことを確かめれば、conversion により $\text{Path } A a c$ の型付けが得られる。0 の側の面を確認しよう:

$$\begin{aligned} (p \cdot q) 0 &=_{\beta} \mathbf{hcomp}_A^j [1_{\mathbb{F}} \mapsto a, 0_{\mathbb{F}} \mapsto q j] (p 0) && (\text{Path}\beta) : \text{代入 } [0/i] \\ &=_{\partial} a && \partial\text{-等値性; Lemma 27} \end{aligned}$$

第一の等号は (Path β) による代入 $[0/i]$ の計算そのもので、face formula $\neg$ は束準同型として作用し、 $(i=0)[0/i] = 1_{\mathbb{F}}$ 、 $(i=1)[0/i] = 0_{\mathbb{F}}$ となる (Lemma 20)。第二の等号は、面が $1_{\mathbb{F}}$ となった組に対する Lemma 30 の ∂ -等値性—側面 a は j に依存しないからその終端 $a[1/j]$ は a —から、Lemma 27 で制限 $1_{\mathbb{F}}$ を外して得られる。1 の側の面も同様である:

$$\begin{aligned} (p \cdot q) 1 &=_{\beta} \mathbf{hcomp}_A^j [0_{\mathbb{F}} \mapsto a, 1_{\mathbb{F}} \mapsto q j] (p 1) && (\text{Path}\beta) : \text{代入 } [1/i] \\ &=_{\partial} q 1 && \partial\text{-等値性; Lemma 27} \\ &=_{\partial} c && \text{端点の } (\partial) \text{ 規則 } (q : \text{Path } A b c) \end{aligned}$$

よって $p \cdot q$ はたしかに a から c への path である。この $\mathbf{hcomp}$ は次の正方形の破線の矢印として視覚化できる:



なお、path の反転は代入だけで得られ、 $\mathbf{hcomp}$ を要しない: $p^{-1} \stackrel{\text{def}}{=} \langle i \rangle p[1-i/i] : \text{Path } A b a$ 。

3.3.1 Computation Rules

hcomp の計算規則は、 hcomp が構文上もつ型注釈の形に関する場合分けで、一段簡約 $\rightarrow_\kappa$ (κ -簡約) として向き付けて与える。以下、各規則の左辺は Definition 29 に従って型付けされているものとする。簡約の前後で型は保たれ、右辺は extent φ 上で $u[1/i]$ に等しい。これらを β とは別の簡約として扱う理由は Section 3.6 で論じる。型が動かないため、いずれの場合も簡明である。

以下、各型ごとに hcomp の計算規則を再帰的に与える。個々の規則の背後にある設計の見取り図は、節末の Remark 36 で整理する。

■**II 型** ($C = (x : A) \rightarrow B$) 引数 v に適用されたとき：

$$(\text{hcomp}_C^i [\varphi \mapsto \mu] \lambda_0) v \rightarrow_\kappa \text{hcomp}_{B[v/x]}^i [\varphi \mapsto \mu v] (\lambda_0 v)$$

型が動かないため、引数 v をそのまま各所へ適用すればよい。

■**自然数型** ($C = \mathbb{N}$) 再帰によって与える：

$$\begin{aligned} \text{hcomp}_{\mathbb{N}}^i [\varphi \mapsto 0] 0 &\rightarrow_\kappa 0 \\ \text{hcomp}_{\mathbb{N}}^i [\varphi \mapsto \mathbf{s}(n)] (\mathbf{s}(n_0)) &\rightarrow_\kappa \mathbf{s}(\text{hcomp}_{\mathbb{N}}^i [\varphi \mapsto n] n_0) \end{aligned}$$

■**Path 型** ($C = \text{Path } A u v$)

$$\text{hcomp}_C^i [\varphi \mapsto p] p_0 \rightarrow_\kappa \langle j \rangle \text{hcomp}_A^i [\varphi \mapsto p j, (j = 0) \mapsto u, (j = 1) \mapsto v] (p_0 j)$$

■**選言型** ($C = A + B$) 底面が入射のどちらであるかで場合分けする。底面が $\iota_1(a_0)$ のとき、側面 w も extent 上で ι_1 の形をしている (φ 上で $w[0/i] = \iota_1(a_0)$) であり、選言型の元は i に沿って入射を変えられないため) から、 $\Gamma, \varphi, i : \mathbb{I} \vdash a : A$ がとれて $w = \iota_1(a)$ となる。このとき

$$\text{hcomp}_{A+B}^i [\varphi \mapsto \iota_1(a)] (\iota_1(a_0)) \rightarrow_\kappa \iota_1(\text{hcomp}_A^i [\varphi \mapsto a] a_0)$$

とし、 ι_2 の場合も同様とする。

■**単位型・空型・列挙型** $\top$ と、相異なる定数 $c_1, \dots, c_n$ を標準形とする列挙型 $\{c_1, \dots, c_n\}$ では、底面の標準形がそのまま結果となる：

$$\text{hcomp}_{\top}^i [\varphi \mapsto u] u_0 \rightarrow_\kappa u_0 \quad \text{hcomp}_{\{c_1, \dots, c_n\}}^i [\varphi \mapsto u] c_l \rightarrow_\kappa c_l$$

$\perp$ には標準形がないので規則は不要である (底面 $u_0 : \perp$ が存在する文脈は矛盾している)。これらの型は「path をもたない」離散的な型であり、 hcomp が底面を素通しにすることがそれを表している。

■ **Σ 型** ($C = (x : A) \times B$) 第 1 成分は hcomp_A^i で合成できるが、第 2 成分の型は、第 1 成分が合成されていくのに沿って動くため、homogeneous でない一般の合成を要する。この場合の規則は Section 3.5 で与える。

Remark 32 (Homogeneous Filling). hcomp は開いた箱の蓋を与えるが、connection を使うと、底面と蓋を結ぶ内部 (充填) も得られる：

$$\Gamma, j : \mathbb{I} \vdash \text{hfill}_A^i [\varphi \mapsto u] u_0 j \stackrel{\text{def}}{=} \text{hcomp}_A^k [\varphi \mapsto u[j \sqcap k/i], (j = 0) \mapsto u_0] u_0 : A$$

これは $j = 0$ で u_0 に、 $j = 1$ で $\text{hcomp}_A^i[\varphi \mapsto u] u_0$ に判断的に等しく、 φ 上では $u[j/i]$ に等しい（検算は直後の Exercise 33 で扱う）。Cubical Agda の `hfill` に対応する操作であり、Section 3.5 の Σ 型の合成規則で用いる。

Exercise 33. Remark 32 の hfill_A^i について、その 3 つの性質— $j = 0$ で u_0 、 $j = 1$ で $\text{hcomp}_A^i[\varphi \mapsto u] u_0$ 、 φ 上で $u[j/i]$ —を、定義 $\text{hcomp}_A^k[\varphi \mapsto u[j \sqcap k/i], (j = 0) \mapsto u_0] u_0$ から確かめよ。（ヒント： $j = 0$ の場合、面 $(j = 0)$ が $1_{\mathbb{F}}$ となることに注意し、Lemma 30 の ∂ -等値性と Lemma 27 を用いよ。）

Exercise 34. 始点を共有する 2 本の $\text{path } p : \text{Path } A \text{ } a \text{ } b$ と $q : \text{Path } A \text{ } a \text{ } c$ に対して

$$r \stackrel{\text{def}}{=} \langle i \rangle \text{hcomp}_A^j [(i = 0) \mapsto p \text{ } j, (i = 1) \mapsto q \text{ } j] a$$

とおく。本文の $p \cdot q$ の検算にならい、Lemma 30 の前提（側面と底面の接続・両立条件）と端点の検算を行って、 $r : \text{Path } A \text{ } b \text{ } c$ を導出せよ。この箱は $p \cdot q$ のときと形が違ふことに注意：底面は定値 a （1 点に退化した線）であり、2 つの側面は同じ点から立ち上がる。対応する正方形の図を描き、 r が「 p を逆走してから q を進む」合成（ $p^{-1} \cdot q$ にあたるもの）であることを確かめよ。また $q \stackrel{\text{def}}{=} 1_a$ と特殊化すると、 r の端点は $p^{-1} \stackrel{\text{def}}{=} \langle i \rangle p[1-i/i]$ （Exercise 16）の端点と一致することを確認せよ（両者が Path の元として等しいかどうかは、ここでは問わない）。

Exercise 35. 型ごとの κ -規則を実際に使ってみよう。（1）任意の extent φ のもとで、側面・底面がともに定値 $\mathbf{s}(\mathbf{s}(0))$ の項

$$\text{hcomp}_{\mathbb{N}}^i[\varphi \mapsto \mathbf{s}(\mathbf{s}(0))](\mathbf{s}(\mathbf{s}(0)))$$

を考える（側面が i を含まないので、Lemma 30 の前提は自明に満たされる）。自然数型の κ -規則を繰り返し適用して、この項を正規形まで簡約せよ（ $\mathbf{s}$ の規則を 2 回、 0 の規則を 1 回使う）。「離散的な型では hcomp は底面を素通しする」こと（直後の Remark 36 で整理する）の最小の実例である。（2）関数型の $\text{path } p : \text{Path } (A \rightarrow B) \text{ } f \text{ } g$ と $q : \text{Path } (A \rightarrow B) \text{ } g \text{ } h$ の合成を、本文の $p \cdot q$ の定義どおり

$$p \cdot q \stackrel{\text{def}}{=} \langle i \rangle \text{hcomp}_{A \rightarrow B}^j [(i = 0) \mapsto f, (i = 1) \mapsto q \text{ } j] (p \text{ } i)$$

とおく。 $v : A$ に対して各点の path を $p_v \stackrel{\text{def}}{=} \langle i \rangle p \text{ } i \text{ } v : \text{Path } B \text{ } (f \text{ } v) \text{ } (g \text{ } v)$ 、 $q_v \stackrel{\text{def}}{=} \langle i \rangle q \text{ } i \text{ } v : \text{Path } B \text{ } (g \text{ } v) \text{ } (h \text{ } v)$ とおくと、

$$\langle i \rangle (p \cdot q) \text{ } i \text{ } v = p_v \cdot q_v$$

が判断的に成り立つこと—関数の path の合成は、各点での合成にほかならないこと—を示せ。ヒント：まず $(p \cdot q) \text{ } i$ を $(\text{Path } \beta)$ で開き、得られた hcomp の v への適用に Π 型の κ -規則を 1 回適用する。その際、 system への引数適用が

$$[(i = 0) \mapsto f, (i = 1) \mapsto q \text{ } j] v = [(i = 0) \mapsto f \text{ } v, (i = 1) \mapsto q \text{ } j \text{ } v]$$

と成分ごとに働くことを、多組の略記を単一組に展開したうえで Definition 26 の規則により確かめて用いる。最後に、得られた項が $p_v \cdot q_v$ の定義（ $p \cdot q$ の定義で A を B 、 $f \cdot q$ を $f \text{ } v \cdot q_v$ と読み替えたもの）の展開と一致することを見ればよい。

Remark 36 (The Three Destinations of `hcomp`). 本節で与えた計算規則の全体を、見取り図として整理しておこう。`hcomp` の計算をどう与えるかは型ごとの設計判断であり、行き先は3通りある。第一に、 $\Pi \cdot \Sigma \cdot \text{Path}$ のような構造をもつ型では、`hcomp` は構造を通して成分へ押し込まれる。これは強制された選択である：たとえば Π 型の標準形が λ 抽象でなければ適用が計算できないから、`hcomp` を Π 型の標準形として残す余地はない。第二に、自然数型・選言型・単位型・列挙型のような、`path` 構成子をもたない帰納型では、`hcomp` は構成子に関する再帰で消去できる。これが可能なのは、これらの型の元が i に沿って構成子を変えられない—側面が `extent` 上でつねに底面と同じ構成子を頭にもつ—からである。こちらは強制ではなく選択であり（`hcomp` を残す設計もありうる。Section 4.1 末尾で触れる）、本書は原論文と Cubical Agda にならって再帰的消去を採る。これらの型の「離散性」はこの規則群に現れる。第三に、Chapter 4 で導入する高次帰納型—円周 S^1 の `loop` のような `path` 構成子をもつ型—では、側面が構成子頭とは限らず、`hcomp` に簡約先が存在しない。そこでは `hcomp` の元それ自体を標準形として受け入れることになる。本節の規則群は、このうち第一・第二の場合を型の形ごとに与えたものである。

3.4 Transport

3.4.1 Why `hcomp` Is Not Enough

Definition 29 にみられる `hcomp` の際立った特徴は、前提と結論に現れる型がすべて同じ A だという点である（homogeneous の名の由来）。底面も側面も蓋も、1つの型 A の中に住んでおり、`hcomp` が生み出すのは A の中の新しい `path`（例： $p \cdot q$ ）である。いいかえると、`hcomp` は出発した型から外へ出られない。

ところが、Section 2.3 末尾で立てた課題においては、型をまたぐ推論が要求される。`idpeel`（J 規則）が果たすべき「等しいものを置き換える」推論とは、最も素朴な形では、 $p : \text{Path } A \ a \ b$ と型族 $x : A \vdash P : \text{type}$ が与えられたとき、 $P[a/x]$ の元から $P[b/x]$ の元を得ること（Leibniz の代入原理）である。前提の住む型 $P[a/x]$ と結論の住む型 $P[b/x]$ は一般に異なる型だから、この推論を `hcomp` で型付けすることはできない。

ここで鍵となるのは、この2つの型が**型の線**で結ばれているという観察である。`universe` を持ち出すまでもなく、型族 P と `path p` を合成すれば、 $(\text{Path } E)$ と代入だけで

$$\Gamma, i : \mathbb{I} \vdash P[p\ i/x] : \text{type}$$

という、方向 i に依存する型—0 端が $P[a/x]$ 、1 端が $P[b/x]$ の線—が得られる。依存型と `path` がある限り、方向に依存する型の線は体系の至るところに現れる。

そして `hcomp` の型付けは、 A が i に依存しないことを要求するため、この役割を果たせない。そこで第2の primitive として、この欠けた能力—型の線 $\Gamma, i : \mathbb{I} \vdash A : \text{type}$ に沿って、0 端の型 $A[0/i]$ の元から 1 端の型 $A[1/i]$ の元を得ること—だけを担う操作、移送（`transport`; `transp`）を導入する。「0 端の元・1 端の元」なら `hcomp` にも現れる（ $p \cdot q$ の検算に出てきた $p\ 0$ や $u[1/i]$ がそれである）が、それらはすべて1つの固定された型 A の住人であった。`transp` では線をなすのが**型そのもの**であり、入力（住む $A[0/i]$ ）と出力（住む $A[1/i]$ ）は一般に異なる型である。側面も蓋もない「箱のない」操作である：型を渡ることだけが `hcomp` にない能力なのだから、それだけを切り出すのである（側面つきの箱に、動く型の中で蓋をする一般の合成は、2つの primitive を組み合わせて Section 3.5 で構成する）。まとめれば、等式には「証明を組み合わせる」働きと「証明を使う」働きの2つがあり、前者—1つの型の中で完結する推移律 $p \cdot q$ など—を `hcomp` が、後者—型を跨ぐ Leibniz—を `transp` が担う。等号型では `idpeel` が一手に引き受け

ていたこの2役を、cubical type theory は2つの primitive に分業させるのである。なお、J の働きを厳密に数えるなら、もう1つ「path の中身に触れる」—path から点の情報を取り出す—という側面がある。こちらは Kan 構造を待つまでもなく、path 適用 pr と端点の ∂ -等値性がどの型でも一様に担っている。Section 2.2 の諸例—とりわけ積型の path の分解—が $hcomp$ も $transp$ も使わずに判断的に成立したのは、このためである。「path が区間からの関数であること」と「Kan 構造」という2つの機構は、独立に効いている。J に対応する除去規則そのものは、本節の末尾で、この2つ（と connection による singleton の可縮性）から再構成される。

Remark 37 (Substitution 規則との関係). ここで述べた Leibniz の代入原理は、等号型の理論で **substitution** と呼ばれる導出可能な規則— $Q : A \rightarrow \text{type}$, $E : M =_A N$ と $Q M$ の証明から $Q N$ の証明を得る。MLTT では motive を $\lambda x.\lambda y.\lambda z.(Q x \rightarrow Q y)$ として idpeel から導出され、証明項は $\text{idpeel}_E(\lambda z.\lambda u.u)$ である—toりわけ積型で置き換えが起こるのは**型の位置**（判断の型 $Q M$ 中の M ）であり、移送されるのはその型の**証明**である。

数学の実践で「等しいものの置き換え」というとき、これとは別の形もある：**項の位置**での置き換え、すなわち項の中の部分項 M を等しい N で置き換えたとき、置き換えの前後を結ぶ**等式そのもの**が得られることである。部分項の位置を関数 $t : A \rightarrow B$ として切り出せば、これは「 $t M = t N$ の証明を得る」こと（合同性）にあたる。MLTT ではこちらも J から導出される（Theorem 1 の証明で用いた cong_s がその実例である）。

cubical type theory では、この2つの「置き換え」の運命が分かれる。項の位置の置き換えは、Kan 構造を待つまでもなく path 適用だけで得られる： $\langle i \rangle t(p i)$ —Section 2.2 の ap —が求める path である。これは上で述べた「path の中身に触れる」働きの側に属する。他方、型の位置の置き換え、すなわち substitution そのものは型を跨ぐ推論であり、まさに $transp$ の仕事である（Exercise 39 で subst として構成する）。MLTT では J という1つの規則の2つの影であったものが、cubical では「どの型でも一様に成り立つ側」と「Kan 構造を要する側」とに分解されるのである。

3.4.2 Definition and Computation Rules

以下、型の線 $\Gamma, i : \mathbb{I} \vdash A : \text{type}$ に沿った移送の規則を与える。

Definition 38 (Transport).

$$\frac{\Gamma \vdash \varphi : \mathbb{F} \quad \Gamma, i : \mathbb{I}, \varphi \vdash A = A[0/i] : \text{type} \quad \Gamma \vdash u_0 : A[0/i]}{\Gamma \vdash \text{transp}^i A \varphi u_0 : A[1/i]}$$

さらに、結果は extent φ 上で u_0 自身に等しい (∂ -等値性) :

$$\Gamma, \varphi \vdash \text{transp}^i A \varphi u_0 =_{\partial} u_0 : A[1/i]$$

face formula φ は「運ばなくてよい範囲」を指定する：第2の前提は φ 上で線 A が定数であることを要求し、defbox 内の ∂ -等値性は φ 上で結果が u_0 自身であることを述べる。なお第2の前提から $\Gamma, \varphi \vdash A[0/i] = A[1/i] : \text{type}$ が従う（したがってこの等式は文脈 $\Gamma, i : \mathbb{I}, \varphi$ でも成り立ち、上の等値性判断は well-formed である）ことに注意。特に $\varphi = 1_{\mathbb{F}}$ のとき、Lemma 27 により制限は外れて $\text{transp}^i A 1_{\mathbb{F}} u_0 = u_0$ となる。 $hcomp$ と同様、 $transp$ も代入のもとで安定である。

Section 3.3 の $hfill$ と同様に、connection を使うと、 $transp$ の入力と出力を結ぶ充填

が得られる (j は fresh) :

$$\Gamma, i : \mathbb{I} \vdash \text{transpFill}^i A \varphi u_0 \stackrel{\text{def}}{=} \text{transp}^j A[i \sqcap j/i] (\varphi \vee (i = 0)) u_0 : A$$

ここで $\Gamma, i : \mathbb{I}, \varphi \vdash A = A[0/i] : \text{type}$ から

$$\Gamma, i : \mathbb{I}, j : \mathbb{I}, \varphi \vee (i = 0) \vdash A[i \sqcap j/i] = A[i \sqcap j/i][0/j] : \text{type}$$

が従うので、右辺の transp は well-formed である。この操作は

$$(\text{transpFill}^i A \varphi u_0)[0/i] = u_0 \quad (\text{transpFill}^i A \varphi u_0)[1/i] = \text{transp}^j A[j/i] \varphi u_0$$

を満たし、誘導される path は φ 上で定値 u_0 である。

Exercise 39. 本節の冒頭で述べた Leibniz の代入原理を構成しよう。型族 $\Gamma, x : A \vdash P : \text{type}$ と $\Gamma \vdash p : \text{Path } A a b$ 、 $\Gamma \vdash d : P[a/x]$ に対して

$$\text{subst}(p, d) \stackrel{\text{def}}{=} \text{transp}^i (P[p i/x]) 0_{\mathbb{F}} d$$

とおく。 $\Gamma \vdash \text{subst}(p, d) : P[b/x]$ を導出せよ。(ヒント：確かめることは2つある。第一に、線の0端 $P[p 0/x]$ が $P[a/x]$ に判断的に等しいこと (端点の (∂) 規則と合同性、conversion)。第二に、Definition 38 の第2前提が $\varphi = 0_{\mathbb{F}}$ のときは自動的に成り立つこと (Definition 26 第3規則の $n = 0$ の場合により、制限 $0_{\mathbb{F}}$ の下ではすべての判断が成り立つ。)

発展： $p \stackrel{\text{def}}{=} 1_a$ のとき、線 $P[1_a i/x] = P[a/x]$ は定数であるが、 $\text{subst}(1_a, d)$ は一般には d に判断的に等しくない。その理由を、この後で与える κ -簡約の発火条件から説明せよ (Section 3.6、および Section 3.7 の $\text{toId}(1_M)$ の議論と同じ現象である)。

transp の計算規則も、(構文上の引数である) 型の線 A の形に関する場合分けで、 κ -簡約として与える。型が動くことに由来する複雑さは、すべてこちらの側に現れる。

■ **自然数型・単位型・空型・列挙型** これらの型は型構成子が引数をとらないので、線 A は i に依存しない。したがって運ぶ必要がない：

$$\text{transp}^i \mathbb{N} \varphi n_0 \rightarrow_{\kappa} n_0 \quad \text{transp}^i \top \varphi u_0 \rightarrow_{\kappa} u_0 \quad \text{transp}^i \{c_1, \dots, c_n\} \varphi c_l \rightarrow_{\kappa} c_l$$

($\perp$ については hcomp と同じ理由で規則を要しない。)

Remark 40 (「移送」という比喩について)。上の規則だけを見ると、「線に沿って元を運ぶ」という説明は大げさに見えるかもしれない。 $\text{transp}^i \mathbb{N} \varphi n_0 \rightarrow_{\kappa} n_0$ は、 $A[0/i]$ の元をもとに $A[1/i]$ の元を直接定義しているだけであり、「運ぶ」過程はどこにも現れないからである。しかしこれは、これらの型では線 A が定数でしかありえない—型構成子が引数をとらないため、 A が i に依存する余地がない—ことの帰結であり、移送が退化する base case にすぎない。一般の場合に「移送」という説明を支えるのは、次の2点である。

第一に、 transp は端点の型の関数ではなく、**線の関数**である。もし transp が「 $A[0/i]$ と $A[1/i]$ から直接定義される」操作であるなら、同じ端点をもつ2本の線は同じ結果を与えるはずだが、そうはならない。Chapter 5 で導入する ua がその典型であり、同値 $e : B \simeq B$ ごとに両端が同じ型 B である線 $\text{ua}(e)$ が得られ、それに沿った transp は e の関数成分を計算する—同じ端点でも、線が違えば移送の結果は違う。引数をとる型構成子 ($+$ ・ Σ ・ Π ・ Path) の規則が、端点ではなく成分の**線**を参照して再帰するのも、結果が線全体に依存することの現れである。

第二に、「運ばれる途中」は構文的に実在する。上で定義した transpFill^i は、 u_0 と $\text{transp}^j A[j/i] \varphi u_0$ とを結ぶ、線 A の上に横たわる依存 path (Section 2.3) を与える。すなわち各元は、始端から終端へ「瞬間移動」するのではなく、線の上の各位置を通過する。実際、後述の Σ の規則は、第 1 成分の充填 $a \stackrel{\text{def}}{=} \text{transpFill}^i A \varphi (\pi_1(w_0))$ に沿って第 2 成分の線 $B[a/x]$ を定めており、この「途中経過」が計算規則そのものの中で使われている。

■**選言型** ($C = A + B$) 線に沿って入射は変わらないから、成分を運ぶだけでよい：

$$\text{transp}^i(A + B) \varphi (\iota_1(a_0)) \rightarrow_{\kappa} \iota_1(\text{transp}^i A \varphi a_0)$$

ι_2 の場合も同様である。

■ **Σ 型** ($C = (x : A) \times B$) 第 1 成分の移送の充填を

$$a \stackrel{\text{def}}{=} \text{transpFill}^i A \varphi (\pi_1(w_0)) \quad (\text{文脈 } \Gamma, i : \mathbb{I}, \text{ 型 } A)$$

とおくと：

$$\text{transp}^i C \varphi w_0 \rightarrow_{\kappa} (\text{transp}^i A \varphi (\pi_1(w_0)), \text{transp}^i B[a/x] \varphi (\pi_2(w_0)))$$

第 2 成分は、第 1 成分が運ばれていくのに沿って動く線 $B[a/x]$ に沿って運ばれる。

■ **Π 型** ($C = (x : A) \rightarrow B$) 引数 $u_1 : A[1/i]$ に適用されたとき。まず u_1 を線 A に沿って「逆向きに」運ぶ：

$$\begin{aligned} w &\stackrel{\text{def}}{=} \text{transpFill}^i A[1-i/i] \varphi u_1 & (\text{文脈 } \Gamma, i : \mathbb{I}, \text{ 型 } A[1-i/i]) \\ v &\stackrel{\text{def}}{=} w[1-i/i] & (\text{文脈 } \Gamma, i : \mathbb{I}, \text{ 型 } A ; v[1/i] = u_1) \end{aligned}$$

これを用いて：

$$(\text{transp}^i C \varphi \lambda_0) u_1 \rightarrow_{\kappa} \text{transp}^i B[v/x] \varphi (\lambda_0 v[0/i])$$

■**Path 型** Path 型の線に沿った移送は、型が動き、かつ端点という側面がある状況であり、 hcomp と transp を組み合わせた一般の合成を要する。次節でこれを定義する。

3.5 Composition as a Derived Operation

型が動き、側面もある一般の合成 (comp) は、 hcomp と transp を組み合わせて**派生的操作**として構成できる。まず補助操作を定義する（前提は transp の規則と同じ）：

$$\Gamma, i : \mathbb{I} \vdash \text{squeeze}^i A \varphi a \stackrel{\text{def}}{=} \text{transp}^j A[i \sqcup j/i] (\varphi \vee (i = 1)) a : A[1/i]$$

これは線分 a の全体を、その終端のファイバー $A[1/i]$ の中へ「絞り込む」操作であり、

$$(\text{squeeze}^i A \varphi a)[0/i] = \text{transp}^j A[j/i] \varphi (a[0/i]) \quad (\text{squeeze}^i A \varphi a)[1/i] = a[1/i]$$

を満たす (φ 上では定値 a)。

Exercise 41. Exercise 33 にならい、 $\text{squeeze}^i A \varphi a$ の 2 つの面 $(\text{squeeze}^i A \varphi a)[0/i]$ と $(\text{squeeze}^i A \varphi a)[1/i]$ を計算し、本文の記述と一致することを確かめよ。また、 $\text{transpFill}^i A \varphi u_0$ の 2 つの面についても同様に確かめよ。

squeeze を用いると、型が動き側面もある一般の合成 (*composition*) が、「側面を squeeze で終端へ絞り、底面を transp で運び、終端の中で hcomp する」操作として定義できる： $\Gamma \vdash \varphi : \mathbb{F}, \Gamma, i : \mathbb{I} \vdash A : \text{type}, \Gamma, \varphi, i : \mathbb{I} \vdash u : A, \Gamma \vdash u_0 : A[0/i]$ (ただし $\Gamma, \varphi \vdash u_0 = u[0/i] : A[0/i]$) に対して

$$\begin{aligned} & \text{comp}^i A [\varphi \mapsto u] u_0 \\ & \stackrel{\text{def}}{=} \text{hcomp}_{A[1/i]}^i [\varphi \mapsto \text{squeeze}^i A 0_{\mathbb{F}} u] (\text{transp}^i A 0_{\mathbb{F}} u_0) : A[1/i] \end{aligned}$$

これは extent 上で側面の終端に等しい (∂ -等値性)：

$$\Gamma, \varphi \vdash \text{comp}^i A [\varphi \mapsto u] u_0 =_{\partial} u[1/i] : A[1/i]$$

(hcomp と squeeze の性質から従う。) 歴史的には、Cohen et al. (2018) はこの comp を primitive として採用し、型ごとの再帰で計算規則を与えた。上の分解は Coquand et al. (2018) による。

Exercise 42. 上で定義した comp について、その底面条件 $\Gamma, \varphi \vdash \text{transp}^i A 0_{\mathbb{F}} u_0 = (\text{squeeze}^i A 0_{\mathbb{F}} u)[0/i] : A[1/i]$ が成り立つことを、 squeeze の面の性質と hcomp の前提から示せ。これにより comp の定義が well-formed であることが確かめられる。

Exercise 43. Section 2.3 で述べた、依存 path と移送を経由した通常の path との対応

$$\text{PathP}^i A u_0 u_1 \longleftrightarrow \text{Path } A[1/i] (\text{transp}^i A 0_{\mathbb{F}} u_0) u_1$$

を構成せよ。(ヒント： $p : \text{PathP}^i A u_0 u_1$ が与えられたとき、 $\text{transpFill}^i A 0_{\mathbb{F}} u_0$ は u_0 から $\text{transp}^i A 0_{\mathbb{F}} u_0$ への A に沿った依存 path であり、 p と「向きを合わせて」合成すれば $A[1/i]$ の中の path が得られる。逆向きの構成も同様に transpFill を用いる。なお、ここでの「合成」は型の線 A の上での 2 本の依存 path の合成であって、 hcomp ではなく comp を要する—この練習問題が comp の後に置かれている理由である。別解として、 $\langle i \rangle \text{squeeze}^i A 0_{\mathbb{F}} (p i)$ が求める path を直接与える： squeeze の 2 つの面の性質から端点を確認せよ。)

comp を用いると、先送りになっていた 2 つの計算規則が書ける。 Σ 型の hcomp ($C = (x : A) \times B$) は、第 1 成分の充填を

$$c \stackrel{\text{def}}{=} \text{hfill}_A^i [\varphi \mapsto \pi_1(w)] (\pi_1(w_0)) i \quad (\text{文脈 } \Gamma, i : \mathbb{I}, \text{ 型 } A)$$

とおいて与える。ここで hfill_A^i の肩の i は側面 $\pi_1(w)$ の中の i を束縛しており、結果には現れない。したがって最後の引数 (充填の径数) に i を渡すことができ、 c は i を径数とする線となる。実際 $c[0/i] = \pi_1(w_0)$ 、 $c[1/i] = \text{hcomp}_A^i [\varphi \mapsto \pi_1(w)] (\pi_1(w_0))$ であり、 c は底面の第 1 成分から合成された第 1 成分へ至る線である。これを用いて：

$$\begin{aligned} & \text{hcomp}_C^i [\varphi \mapsto w] w_0 \\ & \rightarrow_{\kappa} (\text{hcomp}_A^i [\varphi \mapsto \pi_1(w)] (\pi_1(w_0)), \text{comp}^i B[c/x] [\varphi \mapsto \pi_2(w)] (\pi_2(w_0))) \end{aligned}$$

Path 型の transp ($C = \text{Path } A u v ; A, u, v$ は i に依存してよい) は：

$$\text{transp}^i C \varphi p_0 \rightarrow_{\kappa} \langle j \rangle \text{comp}^i A [\varphi \mapsto p_0 j, (j = 0) \mapsto u, (j = 1) \mapsto v] (p_0 j)$$

3.6 Classifying the Computation Rules

型理論の redex 規則は、導入則の直後に除去則を適用する detour の除去として与えられる。 natrec の計算規則も、導入形 $\mathbf{s}(n)$ を除去子が消費する同型の規則である。これに対し、本章の $\text{hcomp} \cdot \text{transp}$ にともなう等値性判断は、この枠に収まらない。

第一に、 hcomp と transp は、どの型構成子の導入則でも除去則でもない。両者はすべての型に様に課される Kan 構造の操作であって、導入則が検証条件を、除去則が使用条件を与えて型の意味を定める、という枠組みの中に席をもたない。第二に、規則が発火するトリガーが項の主構成子ではない。 transp の計算規則は型の線の頭構成子で場合分けされ（型主導の計算）、 hcomp の Π 則は、除去子（関数適用）が canonical でない頭（ hcomp ）に出会ったときに働く可換規則の形をとる— β とはトリガーの向きが逆である。さらに、 $\varphi = 1_{\mathbb{F}}$ や端点 $0, 1$ で発火する等値性は、項の形と無関係に「側面から」働く。第三に、そして決定的なことに、高次帰納型（Chapter 4）では $\text{hcomp}_{\mathbb{Z}}^i$ は構成子であり、その項は canonical form であって、いかなる規則も発火しない。同じ形の項が、型に応じて redex にも値にもなる以上、これらの等値性を特定の型構成子に固有の β として分類することはできない。

そこで本書では、これらの等値性を β と区別したうえで、さらに性格の異なる 2 つの類に分けて扱う。

第一の類は、 $\text{hcomp} \cdot \text{transp}$ が構文上もつ型注釈（高次帰納型では構成子）の形で発火が決まる、Kan 操作の計算規則である。発火が構文的に決まる以上、これらは β と同様に向き付けることができ、本書では一段簡約 $\rightarrow_{\kappa}$ として与えた（ κ -簡約）。計算はこの向きで走り、体系の canonicity (Huber 2019) を支える。

第二の類は、面・端点において成り立つ等値性である： extent 上の等値性（ $\Gamma, \varphi \vdash \text{hcomp} \cdots =_{\partial} u[1/i]$ ）、 path 適用の端点規則（ $t0 =_{\partial} u_0$ ；Definition 13）、高次帰納型の構成子の境界（ $\text{eqmnpqe}0 =_{\partial} [\langle m, n \rangle]$ ；Chapter 4）がこれにあたる。この類は原理的に向き付けることができない。端点規則の右辺 u_0 は左辺の項のどこにも現れず、 t の型からしか読み取れない。また extent 上の等値性は制限文脈 Γ, φ においてのみ成り立つ判断であって、項の上の大域的な書き換えの閉包には入らない。よってこの類は等値性判断のまま残し、由来を示す添字を付けて $=_{\partial}$ と書く（ ∂ -等値性；記号は boundary formula ∂I にちなむ）。

この 2 本立てから、cubical type theory の判断的等値性の姿が確定する。本書は、MLTT については、判断的等値性を β 簡約と η の生成する合同（ $=_{\beta\eta}$ ）として与え、conversion 規則の側条件に用いる立場をとった（Chapter 1）。これに対して **cubical type theory の判断的等値性とは、 β 簡約・ κ -簡約・ η ・ ∂ -等値性（および $\mathbb{I} \cdot \mathbb{F}$ の等式）の生成する合同である**。ただし ∂ -等値性は文脈相対であるため、判断的等値性はもはや項の上の簡約だけでは生成できない。また $\rightarrow_{\kappa}$ と $=_{\partial}$ の間には重なりがあり（たとえば φ が $1_{\mathbb{F}}$ となった κ -redex には両方が適用できる）、 $\rightarrow_{\beta\kappa}$ の合流性と正規化は、 β 簡約の Church–Rosser 性のようには自明でない。この問題は Sterling and Angiuli (2021) により解決されている。

この「型ごとに与えられる計算規則」という形は、Martin–Löf の意味の説明—集合を与えるとは、何がその要素であるかと、どの要素が等しいかを指定することである—to 照らすと見通しがよい。等号型は、後半の「どの要素が等しいか」を、型に依らない一様な構成（ refl を唯一の構成子とする族）で置き換えた。これに対し cubical type theory の Kan 構造は、この後半を型ごとの指定として回復する：「その型で path がどう振る舞うか」を、 $\Pi \cdot \Sigma$ では κ -規則が、高次帰納型では構成子としての hcomp と構成子ごとの transp とが、個別に定めるのである。Chapter 5 で Glue 型の Kan 構造が univalence

を計算可能にするのは、まさに「宇宙という型において、どの要素が等しいかを指定した」ことの帰結である。一様な等号型からは決して出てこない。

なお、Coq などの CIC の伝統では、帰納型の除去子の計算規則を ι -簡約と呼んで β と区別する。本書は `natrec` の計算規則を、導入形を除去子が消費する `detour` の除去、すなわち β の一種として扱う思想を採るため、 ι は導入せず、 β (と η) $\cdot \kappa \cdot \partial$ の分類とする。

3.7 Path Types and the Equality Type

Section 2.3 の末尾で、`path` 型の除去規則 (J に対応する規則) の正当化を課題として立てた。`transp` と Section 2.2 の `singleton` の可縮性を合わせると、等号型について知られているのと同じ仕方でも `Path` 型の除去規則が得られ、この課題は果たされたことになる。これで、`Path` 型が Martin-Löf 型理論の等号型 $M =_A N$ と同じ役割を果たしうることが見えてきた。両者の関係を整理しておこう。

まず記法について。本書では反射律の証明を $1_a \stackrel{\text{def}}{=} \langle i \rangle a$ と書いてきた (Section 2.1)。これは等号型の $\text{refl}_A(a)$ に対応する。ただし両者の由来は異なる： $\text{refl}_A(a)$ は $(=I)$ という導入規則によって与えられる構成子であるのに対し、 1_a は `path` 抽象の特別な場合、すなわち「 i に依存しない定値の線」である。`Path` 型に固有の導入規則はなく、 $\langle i \rangle$ という一つの抽象がすべての `path` を作る。

- **除去規則**：等号型の $(=E)$ (`idpeel`) に対応するものは、Section 2.2 の `singleton` の可縮性と `transp` を組み合わせて `Path` 型に対して構成できる。すなわち `Path` は等号型としての用途を満たす。
- **計算規則の差**：ただし `idpeel` の計算規則 $\text{idpeel}_{\text{refl}_A(M)}^P(R) = RM$ は、`Path` 側では判断的等値性としてではなく `Path` として成り立つにとどまる。これは `Path` 型を採用したときに支払う代価である。
- **関数外延性**：逆に `Path` 型では関数外延性が Section 2.2 のように直接構成できる。等号型では関数外延性は導出できず (Theorem 1)、公理として仮定するか observational type theory のような別の機構を要する。これが `Path` 型を採用して得るものである。
- **UIP と K**：Chapter 1 で触れた uniqueness of identity proofs は、`Path` 型では成り立たない。`Path (Path A a a) 1_a 1_a` が自明でない元をもつ型 A が存在しうるからである (これが「型は集合ではなく高次の構造をもちうる」という Chapter 4 の高次帰納型の出発点である)。一方、Section 4.7 で見るように、特定の型についてはそれが集合であることを証明できる。

■相互変換の明示的な構成 以上の関係を、項のレベルで具体化しよう。本書の体系に、Martin-Löf 型理論の等号型—形成 $(=F)$ ・導入 $(=I)$ ・除去 $(=E)$ とその計算規則—をそのまま追加する*5。

等号型から `path` への変換は、`idpeel` (J 規則) による。motive を $P \stackrel{\text{def}}{=} \lambda x. \lambda y. \lambda _ . \text{Path } A x y$ として

$$\text{toPath}(E) \stackrel{\text{def}}{=} \text{idpeel}_E^P(\lambda x. 1_x) : \text{Path } A M N \quad (E : M =_A N)$$

と定める。 $R \stackrel{\text{def}}{=} \lambda x. 1_x$ が $(x : A) \rightarrow P x x (\text{refl}_A(x)) = (x : A) \rightarrow \text{Path } A x x$ の元で

*5 追加された等号型を Kan にする (`hcomp`・`transp` の κ -簡約を与える) 一般論としては Swan の構成が知られている。Cohen et al. (2018) の §9.1 を参照。以下の構成では等号型の κ -簡約は使わない：`toId` の型付けには Definition 38 の一般の型付け規則だけで足りる。

あることに注意。idpeel の計算規則により

$$\text{toPath}(\text{refl}_A(M)) = 1_M$$

が判断的に成り立つ。

逆向きの変換は、transp による。 $p : \text{Path } A \ M \ N$ に対して、型の線 $\Gamma, i : \mathbb{I} \vdash M =_A p \ i : \text{type}$ を考える ($(=F)$ と $(\text{Path}E)$ による)。その始端は、端点の ∂ -規則 $p \ 0 =_{\partial} M$ により $M =_A M$ に判断的に等しいから、 $\text{refl}_A(M)$ を運ぶことができる：

$$\text{toId}(p) \stackrel{\text{def}}{=} \text{transp}^i (M =_A p \ i) \ 0_{\mathbb{F}} (\text{refl}_A(M)) : M =_A N$$

univalence も Glue も不要であることに注意—Definition 38 の型付け規則だけで型が付く。ただし toPath と違い、toId(1_M) は $\text{refl}_A(M)$ に判断的には等しくない。線 $M =_A 1_M \ i = M =_A M$ は定値だが、この線に対する transp の κ -簡約を与えていないからである。両者を結ぶ path は transFill から得られる：

$$\begin{aligned} \text{tr}_M &\stackrel{\text{def}}{=} \langle i \rangle \text{transpFill}^i (M =_A M) \ 0_{\mathbb{F}} (\text{refl}_A(M)) \\ &: \text{Path } (M =_A M) (\text{refl}_A(M)) (\text{toId}(1_M)) \end{aligned}$$

(transFill の 2 つの面の性質による。)

Theorem 44 (Equality Type and Path Type). 任意の $A : \text{type}$ と $M, N : A$ に対して、以下が成り立つ。

$$\vdash (M =_A N) \simeq (\text{Path } A \ M \ N) \ \text{true}$$

($\simeq$ は Chapter 5 で定義する同値の型である。)

証明の構成：往復が恒等に等しいことを示す。(a) toId $\circ$ toPath の側は、idpeel による。motive を $P' \stackrel{\text{def}}{=} \lambda x. \lambda y. \lambda E. \text{Path } (x =_A y) (\text{toId}(\text{toPath}(E))) \ E$ とすると、 $\text{refl}_A(x)$ の場合の証明義務は $\text{Path } (x =_A x) (\text{toId}(1_x)) (\text{refl}_A(x)) (\text{toPath}(\text{refl}_A(x)) = 1_x$ が判断的に成り立つことを用いた) であり、これは上の tr_x の反転である。(b) toPath $\circ$ toId の側は、Path 型の除去規則 (transp と singleton の可縮性から得たもの) で p を 1_M の場合に帰着させ、toPath(toId(1_M))) と 1_M を、 tr_M に toPath を適用した path $\langle i \rangle \text{toPath}(\text{tr}_M \ i)$ の反転で結ぶ (始端は $\text{toPath}(\text{refl}_A(M)) = 1_M$ 、判断的)。ただし (b) では Path の除去規則の計算規則が Path でしか成り立たないため、 1_M への帰着自体がもう 1 本の path を要する。その合成は hcomp による。互いに逆な写像の組から同値が得られること (Chapter 5) と合わせて、定理を得る。 $\square$

この定理は、等号型を用いて書かれた既存の理論が、Path 型の体系の中でそのまま解釈し直せることを保証する。とくに、(a) の側の往復は等号型の除去だけで済む一方、(b) の側に hcomp が要するという非対称は、idpeel の計算規則が判断的であるのに対し Path の除去のそれが Path 止まりである、という先の対比の再現である。

要約すると、Path 型は等号型の役割を果たしつつ、関数外延性を得る代わりに除去規則の判断的計算規則を失い、UIP を落とす。等号型に基づく意味論—たとえば dependent type semantics (Bekki, 2014)—が Path 型を採用すべきかどうかは、この差引の評価に依存する。

なお cubical type theory は、 Π 型 $\cdot \Sigma$ 型 $\cdot$ path 型について閉じた universe (本書では sort type_i がその役割を担う) と、univalence の証明および universe の合成操作の定義に用いられる Glue 型を備える。これらは Chapter 5 で扱う。

History and Further Reading

simplicial set モデル (Chapter 1 の History) は古典論理に依存しており、また univalence を公理として仮定した型理論は canonicity を失う。構成的なモデルの探求から、Bezem et al. (2014) の cubical set モデルが生まれ、それを構文に移した cubical type theory (Cohen et al., 2018) において、univalence は公理ではなく (Glue 型を用いて証明される) 定理となった。Chapter 2 と本章の枠組み (interval · face lattice · system) は同論文にもとづく。ただし合成操作については、同論文が primitive とする一般の comp をそのまま提示する代わりに、Coquand et al. (2018) に由来する分解にしたがって hcomp と transp を primitive とし (この構成は Cubical Agda (Vezzosi et al., 2019) と一致する)、comp を派生的操作とする提示を採った (Section 3.5)。この体系の canonicity は Huber (2019) により証明されている。なお cubical 型理論には interval に与える代数構造の選択に応じて変種があり、de Morgan 代数に基づく上記の体系のほかに、cartesian cube に基づく計算的型理論 (Angiuli et al., 2017, 2021) がある。また、topos の内部言語による cubical モデルの公理化 (Orton and Pitts, 2016)、正規化定理 (Sterling and Angiuli, 2021) など、メタ理論の研究も進んでいる。

Bibliography

- Altenkirch, T., C. McBride, and W. Swierstra. (2007) “Observational equality, now!”, In *Proceedings of the 2007 Workshop on Programming Languages meets Program Verification (PLPV '07)*, A. Stump and H. Xi (eds.). pp.57–68.
- Angiuli, C., G. Brunerie, T. Coquand, R. Harper, K.-B. Hou (Favonia), and D. R. Licata. (2021) “Syntax and models of Cartesian cubical type theory”, *Mathematical Structures in Computer Science* **31**(4), pp.424–468.
- Angiuli, C., R. Harper, and T. Wilson. (2017) “Computational higher-dimensional type theory”, In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages*. pp.680–693, ACM.
- Awodey, S. and M. A. Warren. (2009) “Homotopy theoretic models of identity types”, *Mathematical Proceedings of the Cambridge Philosophical Society* **146**(1), pp.45–55.
- Bekki, D. (2014) “Representing Anaphora with Dependent Types”, In *Proceedings of Logical Aspects of Computational Linguistics (8th international conference, LACL2014, Toulouse, France), LNCS 8535*, N. Asher and S. V. Soloviev (eds.). pp.14–29, Springer, Heidelberg.
- Bekki, D. and K. Mineshima. (2017) “Context-Passing and Underspecification in Dependent Type Semantics”, In: S. Chatzikyriakidis and Z. Luo (eds.): *Modern Perspectives in Type-Theoretical Semantics*, Vol. 98 of *Studies in Linguistics and Philosophy*. Springer, pp.11–41.
- Bezem, M., T. Coquand, and S. Huber. (2014) “A Model of Type Theory in Cubical Sets”, In *Proceedings of 19th International Conference on Types for Proofs and Programs (TYPES 2013)*, R. Matthes and A. Schubert (eds.), Vol. 26 of *Leibniz International Proceedings in Informatics (LIPIcs)*. pp.107–128, Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
- Cavallo, E. and R. Harper. (2019) “Higher Inductive Types in Cubical Computational Type Theory”, *Proceedings of the ACM on Programming Languages* **3**(POPL), pp.1:1–1:27.
- Cohen, C., T. Coquand, S. Huber, and A. Mörtberg. (2018) “Cubical Type Theory: A Constructive Interpretation of the Univalence Axiom”, In *Proceedings of 21st International Conference on Types for Proofs and Programs (TYPES 2015)*, T. Uustalu (ed.), Vol. 69 of *Leibniz International Proceedings in Informatics (LIPIcs)*. pp.5:1–5:34, Schloss Dagstuhl – Leibniz-Zentrum für Informatik. Preprint: arXiv:1611.02108.
- Coquand, T., S. Huber, and A. Mörtberg. (2018) “On Higher Inductive Types in Cubical Type Theory”, In *Proceedings of LICS '18: Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science*. pp.255–264.
- Hofmann, M. (1995) “Extensional concepts in intensional type theory”, Ph.D. thesis, University of Edinburgh.
- Hofmann, M. and T. Streicher. (1998) “The groupoid interpretation of type theory”,

-
- In: G. Sambin and J. M. Smith (eds.): *Twenty-Five Years of Constructive Type Theory (Venice, 1995)*, Oxford Logic Guides, vol.36. New York, Oxford University Press.
- Huber, S. (2019) “Canonicity for Cubical Type Theory”, *Journal of Automated Reasoning* **63**(2), pp.173–210.
- Kapulkin, K. and P. L. Lumsdaine. (2021) “The simplicial model of Univalent Foundations (after Voevodsky)”, *Journal of the European Mathematical Society* **23**(6), pp.2071–2126.
- Lumsdaine, P. L. and M. Shulman. (2020) “Semantics of higher inductive types”, *Mathematical Proceedings of the Cambridge Philosophical Society* **169**(1), pp.159–208.
- Martin-Löf, P. (1975) “An intuitionistic theory of types”, In: H. E. Rose and J. Shepherdson (eds.): *Logic Colloquium ’73*. Amsterdam, North-Holland, pp.73–118.
- Martin-Löf, P. (1984) *Intuitionistic Type Theory*, Vol. 17. Naples, Italy: Bibliopolis. Sambin, Giovanni (ed.).
- Mörtberg, A. (2021) “Cubical methods in homotopy type theory and univalent foundations”, *Mathematical Structures in Computer Science* **31**(10), pp.1147–1184.
- Nordström, B., K. Petersson, and J. Smith. (1990) *Programming in Martin-Löf’s Type Theory*. Oxford University Press.
- Nordström, B., K. Petersson, and J. M. Smith. (2000) “Martin-Löf’s Type Theory”, In: *Handbook of Logic in Computer Science, Vol 5*. Oxford University Press.
- Orton, I. and A. M. Pitts. (2016) “Axioms for Modelling Cubical Type Theory in a Topos”, In *Proceedings of 25th EACSL Annual Conference on Computer Science Logic (CSL 2016)*, J.-M. Talbot and L. Regnier (eds.), Vol. 62 of *Leibniz International Proceedings in Informatics (LIPIcs)*. pp.24:1–24:19, Schloss Dagstuhl – Leibniz-Zentrum für Informatik. Journal version: *Logical Methods in Computer Science* 14(4:23), 2018.
- Sterling, J. and C. Angiuli. (2021) “Normalization for Cubical Type Theory”, In *Proceedings of 36th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS 2021)*. pp.1–15, IEEE.
- Streicher, T. (1993) “Investigations into Intensional Type Theory”, Ph.D. thesis, Habilitationsschrift, Ludwig-Maximilians-Universität.
- The Univalent Foundations Program. (2013) *Homotopy Type Theory: Univalent Foundations of Mathematics*. <https://homotopytypetheory.org/book>.
- Vezzosi, A., A. Mörtberg, and A. Abel. (2019) “Cubical Agda: A Dependently Typed Programming Language with Univalence and Higher Inductive Types”, *Proceedings of the ACM on Programming Languages* **3**, pp.87:1–87:29.